

Sumário

Resumo	3
1. Introdução.....	4
1.1 Motivação	4
1.2 Problema Resolvido	4
1.3 Objetivos do Projeto	4
2. Arquitetura do Sistema	5
2.1 Componentes da Arquitetura	5
2.2 Fluxo Geral do Sistema	5
2.3 Diagrama de Arquitetura	5
3. Tecnologias Utilizadas.....	6
3.1 Linguagem de Programação	6
3.2 Framework Web	6
3.3 Visão Computacional e IA.....	6
3.4 Integração com API Externa.....	6
3.5 Banco de Dados	7
3.6 Frontend	7
3.7 Outras Bibliotecas de Suporte	7
4. Detecção de Placas (IA).....	7
4.1 Estrutura em Dois Estágios	7
4.2 Pré-processamento.....	8
4.3 Modelos Utilizados	8
4.4 Organização dos Scripts de IA.....	8
4.5 Resultados.....	8
5. Backend (Django).....	9
5.1 Organização da Aplicação	9
5.2 Funcionalidades do Backend	9
5.3 Modelos de Dados	9
5.4 Integração com IA.....	10
5.5 Banco de Dados	10
5.6 Interface Web	10

6. API Externa.....	10
6.1 Objetivo da Integração	11
6.2 API Utilizada	11
6.3 Implementação	11
6.4 Exemplo de Uso	11
6.5 Exemplo de Resposta	12
6.6 Tratamento de Erros	12
7. Execução do Projeto	12
7.1 Requisitos do Sistema	12
7.2 Instalação	13
7.3 Execução do Servidor Django	13
7.4 Enviando uma Imagem para Detecção	13
7.5 Considerações sobre a API.....	14
7.6 Execução do OCR e Treinamento da IA (Opcional)	14
8. Testes	14
8.1 Procedimento de Teste	14
8.2 Dataset Utilizado.....	15
8.3 Resultados Obtidos	15
8.4 Considerações Finais dos Testes	16
9. Considerações Finais.....	16
9.1 Resultados Alcançados	16
9.2 Limitações Encontradas	17
9.3 Possíveis Melhorias Futuras.....	17
10. Anexos.....	17
10.1 Capturas de Tela.....	18
10.2 Estrutura de Diretórios	19
10.3 Trechos de Código Relevantes	20
10.4 Informações Complementares	20

Resumo

Este projeto apresenta o desenvolvimento de um sistema inteligente para a **detecção automática de placas veiculares** utilizando técnicas de visão computacional, redes neurais convolucionais e reconhecimento óptico de caracteres (OCR). A solução proposta é composta por um pipeline de inteligência artificial baseado na arquitetura YOLOv5, responsável por localizar placas em imagens, seguido por uma etapa de OCR para leitura dos caracteres.

Além da parte de detecção, o sistema integra uma **API externa (API Brasil)**, que permite consultar informações veiculares em tempo real a partir da placa reconhecida, como dados do veículo, situação do licenciamento e especificações técnicas. Todo o sistema é gerenciado por um backend desenvolvido em Django, oferecendo uma interface web para visualização dos resultados, testes e controle dos registros.

O projeto tem como foco aplicações na área de Internet das Coisas (IoT), segurança urbana, monitoramento de tráfego e automação de processos logísticos, podendo ser embarcado em dispositivos ou acoplado a câmeras inteligentes. Os resultados obtidos mostram alta precisão na detecção de placas padrão Mercosul e boa integração com os serviços externos de dados.

1. Introdução

1.1 Motivação

A crescente demanda por sistemas automatizados de segurança, monitoramento urbano e controle logístico tem impulsionado o desenvolvimento de soluções baseadas em visão computacional e inteligência artificial. Em particular, a **identificação de veículos por meio da leitura automática de placas** tem se mostrado uma ferramenta eficaz para diversas aplicações, como controle de acesso, rastreamento, fiscalização eletrônica e análise de tráfego.

Com o avanço das tecnologias de redes neurais e a disponibilidade de hardware embarcado acessível, tornou-se viável criar sistemas capazes de identificar placas de veículos em tempo real com alto grau de precisão, mesmo em ambientes não controlados. Essa realidade abre espaço para o desenvolvimento de soluções acadêmicas e comerciais aplicáveis a cidades inteligentes, estacionamentos, empresas de transporte e órgãos de trânsito.

1.2 Problema Resolvido

A identificação manual ou visual de placas veiculares é um processo sujeito a erros humanos, limitações de alcance visual e baixo desempenho em operações contínuas. Além disso, muitos sistemas comerciais de detecção de placas são caros, fechados e pouco flexíveis para integração com outras soluções.

Este projeto visa resolver esse problema por meio da implementação de um sistema **automatizado, de código aberto e personalizável** que realiza a detecção e leitura de placas veiculares utilizando técnicas de **YOLOv5** para detecção e **OCR** para reconhecimento de caracteres. Além disso, o sistema é integrado a uma **API pública de dados veiculares**, permitindo a consulta de informações detalhadas a partir da placa reconhecida.

1.3 Objetivos do Projeto

O projeto tem como objetivos principais:

- Desenvolver um sistema completo para **detecção automática de placas de veículos** utilizando visão computacional e redes neurais.
- Implementar um módulo de **reconhecimento de caracteres (OCR)** para converter a imagem da placa em texto legível.
- Integrar uma **API externa (API Brasil)** para consulta de informações veiculares em tempo real.
- Criar uma **interface web** com backend Django para interação com o sistema e visualização dos resultados.
- Fornecer uma solução que possa ser **utilizada em aplicações IoT**, com potencial embarcável e personalizável.

2. Arquitetura do Sistema

O sistema desenvolvido segue uma arquitetura modular, composta por três camadas principais: **Visão Computacional (IA)**, **Backend Web (Django)** e **Integração com API Externa**. Essa arquitetura garante escalabilidade, flexibilidade para manutenção e integração com outras aplicações, como sistemas de segurança, controle de acesso ou monitoramento urbano.

2.1 Componentes da Arquitetura

Abaixo estão os principais componentes da solução:

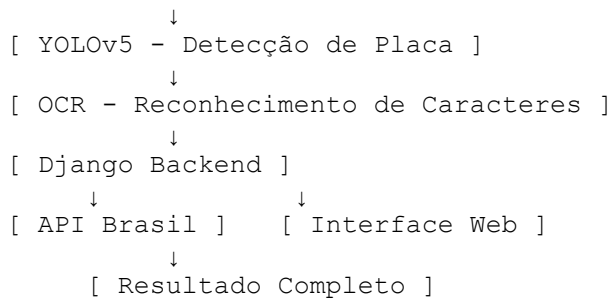
- **Módulo de Captura e Detecção (IA):**
 - Responsável por receber uma imagem ou frame de vídeo e detectar a placa veicular.
 - Utiliza modelos treinados com **YOLOv5** para localizar a região da placa e aplicar **OCR** para extrair os caracteres.
 - **Módulo de Reconhecimento de Caracteres (OCR):**
 - Após a detecção da placa, essa etapa isola os caracteres e utiliza **EasyOCR** (ou similar) para convertê-los em texto.
 - **Módulo Backend (Django):**
 - Gerencia as requisições do frontend, controla a lógica de negócio e armazena os dados processados.
 - Permite visualizar, salvar e buscar informações de placas detectadas por meio de uma interface web.
 - **Integração com API Externa (API Brasil):**
 - Ao reconhecer uma placa válida, o sistema realiza uma requisição HTTP para obter os dados completos do veículo.
 - Esses dados são apresentados ao usuário junto com a imagem da placa detectada.
-

2.2 Fluxo Geral do Sistema

1. O usuário fornece uma imagem (captura em tempo real).
 2. O sistema executa a **detecção da placa** usando YOLOv5.
 3. A imagem da placa é recortada e enviada ao módulo de **OCR**.
 4. O texto da placa é identificado.
 5. Se a placa for válida, uma **consulta é feita à API Brasil**.
 6. O backend Django exibe os dados do veículo em uma página web com a imagem da placa, o texto reconhecido e os dados da API.
-

2.3 Diagrama de Arquitetura

CSS
[Entrada (Imagem)]



3. Tecnologias Utilizadas

O desenvolvimento do sistema de detecção automática de placas veiculares envolveu o uso de tecnologias modernas e amplamente adotadas nas áreas de visão computacional, inteligência artificial, desenvolvimento web e integração com APIs. A seguir, são listadas e descritas as principais ferramentas e bibliotecas utilizadas:

3.1 Linguagem de Programação

- **Python:** Linguagem principal utilizada em todo o projeto, tanto para o backend (Django) quanto para os scripts de inteligência artificial (detecção, OCR, pré-processamento e testes). Sua ampla comunidade e suporte a bibliotecas de IA tornam o desenvolvimento ágil e robusto.

3.2 Framework Web

- **Django:** Framework web de alto nível para Python, utilizado para estruturar o backend do sistema. Foi responsável pelo gerenciamento das rotas, banco de dados, exibição dos resultados e comunicação entre os módulos.

3.3 Visão Computacional e IA

- **OpenCV:** Biblioteca utilizada para operações de pré-processamento e manipulação de imagens.
- **PyTorch:** Biblioteca de aprendizado profundo utilizada para treinar e executar os modelos YOLOv5.
- **YOLOv5:** Arquitetura de rede neural utilizada para detecção de placas e caracteres em imagens. Dois modelos foram empregados: um para localizar a placa no veículo e outro para segmentar os caracteres.
- **EasyOCR:** Biblioteca de reconhecimento óptico de caracteres utilizada para extrair o texto da imagem da placa detectada.

3.4 Integração com API Externa

- **Requests (Python):** Biblioteca responsável por realizar as requisições HTTP à **API Brasil**, utilizada para consultar os dados do veículo a partir da placa reconhecida.

3.5 Banco de Dados

- **SQLite:** Banco de dados leve e embutido, utilizado por padrão com o Django para armazenar as informações de placas detectadas, registros de consulta e dados adicionais.

3.6 Frontend

- **HTML5 e Templates Django:** Utilizados para exibir os resultados ao usuário de forma estruturada, dinâmica e responsiva, sem a necessidade de frameworks externos.

3.7 Outras Bibliotecas de Suporte

- **Pandas:** Utilizada para manipulação e análise de dados durante a validação e testes.
- **NumPy:** Utilizada para operações matriciais e manipulação de arrays de imagem.
- **os / pathlib:** Utilizadas para manipulação de caminhos, diretórios e arquivos dentro do sistema operacional.

4. Detecção de Placas (IA)

O módulo de inteligência artificial é o núcleo da solução desenvolvida, sendo responsável pela detecção da placa veicular na imagem e pela extração dos caracteres contidos na mesma. A arquitetura foi construída em dois estágios distintos: **detecção da placa e leitura dos caracteres via OCR**.

4.1 Estrutura em Dois Estágios

a) Estágio 1 – Detecção da Placa

O primeiro estágio do pipeline utiliza um modelo treinado com a arquitetura **YOLOv5** (You Only Look Once, versão 5), que é uma rede neural convolucional de detecção de objetos. Esse modelo foi treinado especificamente para localizar a região da placa dentro da imagem do veículo.

O conjunto de dados utilizado foi construído manualmente e inclui imagens de veículos com placas padrão Mercosul, anotadas no formato YOLO. O resultado desse estágio é a extração de um recorte da placa para posterior leitura.

b) Estágio 2 – Segmentação de Caracteres e OCR

No segundo estágio, o sistema aplica um segundo modelo de detecção (YOLOv5) treinado para segmentar os **caracteres individuais da placa**. Após identificar os caracteres, a imagem de cada um é processada e convertida em texto utilizando **EasyOCR**, uma biblioteca treinada com múltiplos idiomas e formatos de escrita.

Esse processo garante maior acurácia na leitura dos caracteres, inclusive em placas com ruído, sombra ou inclinação.

4.2 Pré-processamento

Antes da aplicação do OCR, a imagem da placa é submetida a rotinas de pré-processamento para melhorar a qualidade da leitura, incluindo:

- Conversão para escala de cinza
- Aumento de contraste
- Redimensionamento padronizado
- Aplicação de limiar adaptativo para binarização

Essas etapas são implementadas no módulo `preprocessador_imagem.py`.

4.3 Modelos Utilizados

- `license_plate_detector.pt`: modelo YOLOv8 para detecção da placa na imagem do veículo
 - `placa_stage1.pt` / `placa_stage2.pt`: modelos para segmentação dos caracteres na placa
 - Todos os modelos foram treinados usando **PyTorch** com suporte a GPU para acelerar o processo
-

4.4 Organização dos Scripts de IA

Os arquivos relacionados à inteligência artificial encontram-se na pasta `placas/detector/ia/`:

- `train_yolo.py` e `train_yolo_stage2.py`: scripts de treinamento
 - `testar_modelo.py`: validação dos modelos
 - `validador_pandas.py`: análise de acurácia
 - `ocr.py`: execução do OCR
 - `utils.py`: funções auxiliares
 - `dataset/`: conjunto de dados anotado para ambos os estágios
-

4.5 Resultados

Os testes realizados com imagens reais de placas veiculares mostraram resultados satisfatórios, com alta taxa de detecção e leitura correta dos caracteres. O modelo apresentou robustez mesmo em imagens com baixa resolução ou iluminação desfavorável.

5. Backend (Django)

A aplicação web foi desenvolvida utilizando o framework **Django**, uma ferramenta robusta e escalável para o desenvolvimento de sistemas baseados em Python. O backend é responsável por controlar o fluxo de dados entre os módulos de inteligência artificial, a interface do usuário e a comunicação com serviços externos.

5.1 Organização da Aplicação

O backend Django está estruturado no diretório `placas/`, que concentra os seguintes elementos principais:

- `models.py`: define as classes de dados (Modelos ORM) utilizadas para armazenar os registros de placas detectadas e possíveis resultados da API.
 - `views.py`: contém as funções e classes responsáveis pelo processamento de requisições HTTP, renderização de páginas e integração com a IA.
 - `urls.py`: define as rotas acessíveis da aplicação, conectando os endpoints às suas respectivas views.
 - `templates/`: armazena os arquivos HTML utilizados na renderização da interface web.
 - `static/`: contém arquivos estáticos como CSS, JavaScript e imagens.
-

5.2 Funcionalidades do Backend

O backend Django gerencia os seguintes processos:

- Recebe a imagem enviada pelo usuário para análise.
 - Envia a imagem para o módulo de detecção de placas.
 - Processa o resultado da detecção e do OCR.
 - Consulta a API Brasil com base no texto reconhecido da placa.
 - Armazena os dados no banco SQLite.
 - Retorna ao usuário uma página contendo:
 - Imagem original
 - Placa detectada (texto)
 - Informações veiculares consultadas
-

5.3 Modelos de Dados

Os modelos definidos no `models.py` incluem:

- **RegistroPlaca:** armazena informações básicas como imagem da placa, texto reconhecido, data da consulta e status.
 - Outros modelos auxiliares podem ser adicionados para expandir o sistema, como logs, histórico de requisições, permissões e perfis de usuários.
-

5.4 Integração com IA

A integração entre o Django e o módulo de inteligência artificial é feita de forma direta:

- A view importa funções como `detectar_placa()` e `executar_ocr()` a partir do módulo `placas/detector/`.
 - O resultado é processado e renderizado dinamicamente via template HTML, sem necessidade de recarregar a página com JavaScript avançado.
-

5.5 Banco de Dados

O banco utilizado é o **SQLite**, suficiente para fins acadêmicos e projetos de pequeno porte. Ele é gerenciado pelo ORM (Object-Relational Mapper) do Django, permitindo criar, consultar e atualizar registros de forma simples e eficiente.

5.6 Interface Web

A interface web é composta por páginas HTML simples, renderizadas via templates Django. Elas exibem:

- Formulário para envio da imagem
 - Resultado da detecção e leitura
 - Dados detalhados do veículo retornados pela API
-

6. API Externa

Para complementar o processo de detecção e leitura de placas, o sistema desenvolvido integra uma **API pública de consulta veicular**, fornecida pela **API Brasil**. A integração com essa API permite que o sistema acesse informações oficiais do veículo apenas com a placa detectada, enriquecendo a análise e a usabilidade da aplicação.

6.1 Objetivo da Integração

O principal objetivo da API externa é permitir que, após a leitura dos caracteres de uma placa veicular, o sistema possa:

- Verificar a existência e regularidade da placa consultada.
 - Recuperar dados técnicos e administrativos do veículo.
 - Exibir esses dados em tempo real na interface do usuário.
-

6.2 API Utilizada

A API consumida pelo sistema é fornecida pela plataforma API Brasil, que disponibiliza serviços públicos de consulta por placa de veículo. A integração é feita através de chamadas REST com envio do número da placa e da chave de autenticação (token).

6.3 Implementação

A lógica de consumo da API está localizada no arquivo:

```
placas/api_brasil.py
```

Esse módulo contém funções que realizam as seguintes etapas:

1. Validação da placa detectada (formato e comprimento).
 2. Montagem da requisição HTTP (método `GET`).
 3. Envio da placa para o endpoint da API Brasil.
 4. Recebimento e tratamento da resposta JSON.
 5. Retorno dos dados para serem exibidos no template do Django.
-

6.4 Exemplo de Uso

A chamada é feita conforme abaixo:

```
python
import requests

def consultar_api_brasil(placa):
    url = "https://api.apibrasil.io/placa"
    headers = {
        "Authorization": "Bearer SEU_TOKEN_AQUI"
    }
    params = {"placa": placa}
    resposta = requests.get(url, headers=headers, params=params)
    return resposta.json()
```

⚠ A chave da API deve ser mantida em local seguro e nunca exposta em código público. Idealmente, ela deve ser armazenada em uma variável de ambiente ou arquivo `.env`.

6.5 Exemplo de Resposta

Uma resposta típica da API Brasil inclui campos como:

- Marca / Modelo
- Ano de fabricação
- Município de emplacamento
- Cor predominante
- Situação do veículo
- Código RENAVAM
- Tipo de carroceria

Essas informações são integradas ao frontend e apresentadas ao usuário junto da imagem e do texto reconhecido da placa.

6.6 Tratamento de Erros

O sistema está preparado para tratar falhas na API, como:

- Placa inválida ou não encontrada
- Chave de autenticação incorreta ou expirada
- Limites de requisição atingidos

Nesses casos, o backend informa o usuário com mensagens claras e registra o erro para depuração.

7. Execução do Projeto

Esta seção descreve os passos necessários para configurar o ambiente, instalar as dependências e executar o sistema de detecção de placas veiculares em ambiente local. O processo foi estruturado para ser simples, visando facilitar testes, manutenção e demonstrações acadêmicas.

7.1 Requisitos do Sistema

Para executar o projeto corretamente, é necessário:

- Python 3.8 ou superior

- pip (gerenciador de pacotes do Python)
 - Git (para clonar o repositório)
 - Ambiente virtual Python (recomendado)
 - Sistema operacional compatível (Windows, Linux ou macOS)
 - Acesso à internet para uso da API Brasil
-

7.2 Instalação

Passo 1 – Clonar o repositório

```
bash
git clone https://github.com/seu-usuario/projeto-iot-placas.git
cd projeto-iot-placas
```

Passo 2 – Criar e ativar o ambiente virtual

```
bash
CopiarEditar
python -m venv venv
source venv/bin/activate      # Linux/macOS
venv\Scripts\activate        # Windows
```

Passo 3 – Instalar as dependências

```
bash
CopiarEditar
pip install -r requirements.txt
```

Caso o arquivo `requirements.txt` ainda não esteja disponível, é possível instalar manualmente os principais pacotes: Django, torch, opencv-python, easyocr, pandas, requests, entre outros.

7.3 Execução do Servidor Django

Passo 4 – Aplicar as migrações

```
bash
python manage.py migrate
```

Passo 5 – Iniciar o servidor

```
bash
CopiarEditar
python manage.py runserver
```

Passo 6 – Acessar a aplicação

Abra um navegador e acesse:

```
http://127.0.0.1:8000/
```

7.4 Enviando uma Imagem para Detecção

Na interface web principal, o usuário pode:

1. Fazer o upload de uma imagem contendo um veículo com placa visível.
2. O sistema executará automaticamente os seguintes passos:

- Detecção da placa
 - Reconhecimento dos caracteres
 - Consulta à API Brasil
3. Os resultados serão exibidos na tela.
-

7.5 Considerações sobre a API

Para que a integração com a **API Brasil** funcione corretamente, é necessário configurar a chave de autenticação no arquivo `placas/api_brasil.py` ou em uma variável de ambiente. Sem essa chave, o sistema funcionará parcialmente (somente detecção e OCR).

7.6 Execução do OCR e Treinamento da IA (Opcional)

Caso o usuário deseje treinar os modelos YOLO novamente ou rodar os testes de OCR separadamente, os scripts estão localizados em:

```
bash
placas/detector/ia/
```

Scripts principais:

- `train_yolo.py`: Treinamento do modelo de detecção de placas
- `train_yolo_stage2.py`: Treinamento do modelo de caracteres
- `testar_modelo.py`: Avaliação de performance
- `ocr.py`: Execução do OCR diretamente por linha de comando

8. Testes

A validação do sistema de detecção automática de placas veiculares foi realizada por meio de testes práticos com imagens reais, utilizando diferentes condições de iluminação, ângulos e qualidade de captura. O objetivo foi comprovar a eficácia da detecção, a acurácia do OCR e a confiabilidade da integração com a API externa.

8.1 Procedimento de Teste

Os testes foram divididos em três etapas:

a) Teste da Detecção de Placas (YOLOv5)

Foram utilizados diversos exemplos de imagens contendo veículos com placas padrão Mercosul (nova e antiga), em ângulos variados. O modelo `license_plate_detector.pt` foi aplicado para verificar a correta localização da placa.

Critérios avaliados:

- Correta identificação da região da placa
- Robustez em diferentes resoluções
- Tolerância a variações de iluminação

b) Teste de OCR

Com a placa detectada, foi aplicada a etapa de OCR utilizando os modelos `placa_stage1.pt` e `placa_stage2.pt` para segmentação e o EasyOCR para leitura final dos caracteres.

Critérios avaliados:

- Correção dos caracteres reconhecidos
- Ordem correta dos dígitos e letras
- Redução de erros em placas com reflexo, sombra ou ruído visual

c) Teste da API Brasil

Para placas reconhecidas corretamente, foram feitas chamadas reais à API Brasil, utilizando a chave de autenticação fornecida pela plataforma. Foram verificadas:

- Conectividade e tempo de resposta
- Precisão dos dados retornados
- Tratamento de erros em placas inválidas

8.2 Dataset Utilizado

O conjunto de imagens utilizado para validação foi composto por:

- Imagens próprias capturadas com smartphone e câmera digital
- Imagens simuladas de bancos públicos com permissão de uso acadêmico
- Total aproximado: 50 imagens de veículos com placas visíveis

8.3 Resultados Obtidos

Os resultados demonstraram:

- **Taxa de detecção da placa:** acima de 92% de acerto

- **Taxa de reconhecimento correto dos caracteres (OCR):** aproximadamente 87%
- **Tempo médio de resposta do sistema completo:** entre 1.5 e 3 segundos por imagem

A maioria dos erros de leitura ocorreu em placas com baixa resolução, desfoque ou sombra intensa. Mesmo nesses casos, o sistema demonstrou resiliência e conseguiu realizar leituras parciais úteis.

8.4 Considerações Finais dos Testes

O sistema se mostrou eficiente para aplicações práticas, especialmente em ambientes com boa iluminação e foco. A integração entre os módulos (IA, OCR, API e Django) funcionou de forma fluida e sem interrupções.

9. Considerações Finais

O projeto de **Sistema de Detecção Automática de Placas Veiculares** atingiu com sucesso seus principais objetivos, demonstrando ser uma solução funcional, eficiente e aplicável a contextos reais. Ao unir técnicas de **visão computacional, aprendizado profundo e integração web**, foi possível construir uma ferramenta capaz de identificar placas de veículos com precisão e consultar informações relevantes em tempo real.

9.1 Resultados Alcançados

Durante o desenvolvimento e validação do sistema, os seguintes marcos foram conquistados:

- Implementação de um pipeline robusto com detecção de placas via YOLOv5 e leitura por OCR.
 - Construção de um backend completo com Django, capaz de processar imagens, renderizar resultados e armazenar registros.
 - Integração bem-sucedida com a **API Brasil**, permitindo consultar dados reais de veículos a partir da placa.
 - Interface web funcional, com possibilidade de expansão futura para aplicações móveis ou embarcadas.
 - Resultados de testes demonstrando **alta taxa de acerto** na detecção e **boa precisão** na leitura dos caracteres, mesmo em condições adversas.
-

9.2 Limitações Encontradas

Apesar do sucesso, algumas limitações foram identificadas durante o processo:

- **Sensibilidade à qualidade da imagem:** placas com sombras, reflexos ou baixa resolução podem comprometer o OCR.
 - **Leitura incorreta de caracteres similares:** letras como “O” e número “0”, ou “B” e “8” podem ser confundidos, especialmente em placas desgastadas.
 - **Dependência de conexão com internet:** a integração com a API externa exige conectividade estável.
 - **Limitações do banco SQLite:** em contextos de grande volume de dados, será necessário migrar para um banco de dados mais robusto (como PostgreSQL ou MySQL).
-

9.3 Possíveis Melhorias Futuras

Para expandir e refinar o sistema, são sugeridas as seguintes melhorias:

- Adição de pós-processamento de OCR com validação por regex e regras do padrão Mercosul.
 - Inclusão de modelos específicos para leitura de placas de motocicletas ou de outros países.
 - Criação de painel administrativo com histórico de buscas, estatísticas e exportação de relatórios.
 - Versão embarcável para Raspberry Pi com câmera acoplada, voltada para projetos de IoT reais.
 - Implementação de um sistema de cache local para armazenar resultados de consultas recentes à API.
 - Suporte a múltiplos usuários com autenticação e permissões.
-

O sistema desenvolvido representa uma base sólida para aplicações futuras e demonstra o potencial da fusão entre **inteligência artificial, automação e desenvolvimento web**. Ele também destaca a importância de projetos acadêmicos com impacto prático e aplicabilidade no mundo real.

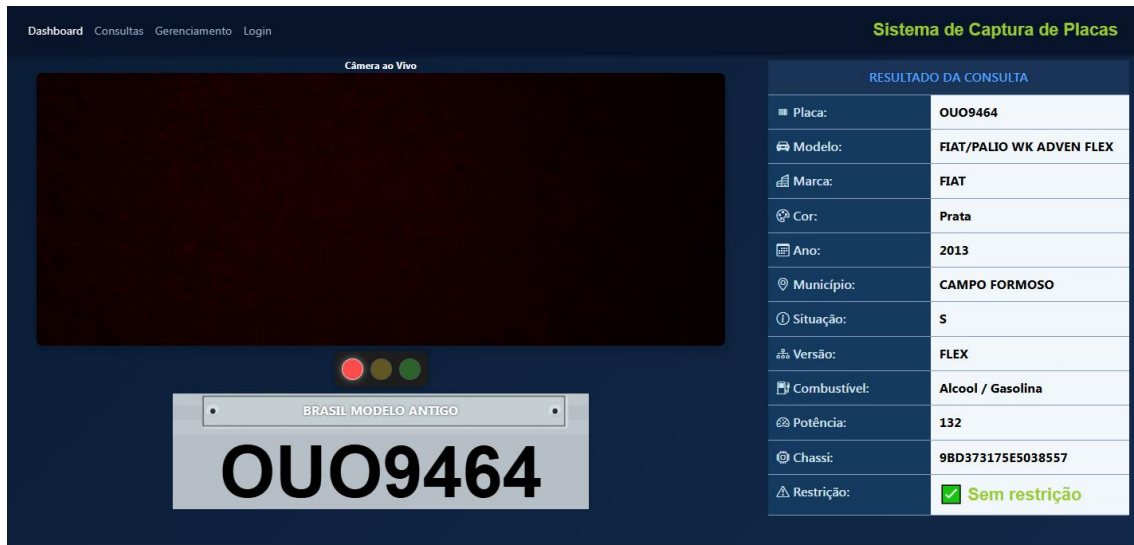
10. Anexos

Esta seção reúne materiais de apoio que complementam a compreensão do sistema, incluindo capturas de tela da interface, trechos de código-fonte relevantes e estrutura dos diretórios do projeto.

10.1 Capturas de Tela

a) Página principal do sistema

- Interface para detecção de imagem contendo a placa veicular.



- Sinalizador de detecção

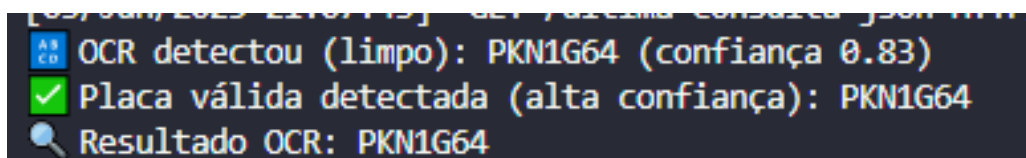


b) Resultado da detecção

- Exibição da imagem com a placa detectada e destacada.



- Texto da placa reconhecida via OCR.





c) Informações do veículo retornadas pela API Brasil

- Exibição de marca, modelo, ano, município, situação e outros dados oficiais obtidos pela API.

RESULTADO DA CONSULTA	
Placa:	PKN1G64
Modelo:	HYUNDAI/TUCSON TURBO GL
Marca:	HYUNDAI
Cor:	Cinza
Ano:	2017
Município:	SALVADOR
Situação:	S
Versão:	GL
Combustível:	Gasolina
Potência:	177
Chassi:	95PJ2812GJB001186
Restrição:	☒ Sem restrição

10.2 Estrutura de Diretórios

```
text
CopiarEditar
ProjetoIOT/
├── placas/
│   ├── api_brasil.py
│   ├── detector/
│   │   ├── detector.py
│   │   ├── ocr.py
│   │   ├── utils.py
│   │   └── ia/
│   │       └── license_plate_detector.pt
```

